00. IIIT RESEARCH PROJECT MASTERY GUIDE

Graph Digital Twin for Smart Grid Cyber-Resilience & Explainable AI

Project: India–Norway INTPART ResCri Initiative — **Institution:** Dr. SPM IIIT Naya Raipur
Researcher: Anshika Keshari (KIIT University, B.Tech CSE) — **Target:** Technical Interview / Viva Defense

How to Use This Master Document

This document is designed for **zero-knowledge comprehension**. Even if you haven't reviewed electrical engineering or graph neural networks in months, every single concept is explained through an **everyday real-world analogy first**, followed by **exact mathematical and architectural rigor**, and concluding with **bulletproof interview Q&A talking points**.

1. The 60-Second Elevator Pitch & Key Numbers

The 60-Second Spoken Pitch (Memorize This!)

"In my research internship at IIIT Naya Raipur under the Indo-Norwegian INTPART ResCri project, I engineered a **Graph Digital Twin** to defend smart electrical power grids against stealthy False Data Injection Attacks (FDIAs).
Traditional grid security relies on Weighted Least Squares (WLS) state estimation, which fails against sophisticated attackers who manipulate sensor telemetry while respecting Kirchhoff's physical laws.
To solve this, I modeled the power grid as a **spatial admittance graph** and deployed a multi-task **GraphSAGE neural network**. My framework achieves a **98.25% detection F1 score** and **97.62% substation localization F1 score** at a sub-millisecond inference latency of **0.908 ms**.
Furthermore, I integrated **Integrated Gradients (XAI)** to provide operators with transparent sensor-level attribution heatmaps, and implemented an automated **closed-loop self-healing module** that mutes corrupted sensors and substitutes historical baseline forecasts, reducing grid state estimation error by **99.97%**."

The 6 Critical Project Numbers You Must Never Forget

Metric	Exact Value	Engineering Significance
Detection F1	98.25% ± 0.34%	5-seed statistical average on IEEE 14-bus system (ROC-AUC: 0.9993).
Localization F1	97.62% ± 0.84%	Pinpoints exact hacked substations among all 14 grid buses.
Inference Latency	0.908 ms	Sub-millisecond execution, well within SCADA's 2–4 second scan cycle.
Outage Generalization	92.80% Det F1	Zero-shot adaptation to $N-1$ transmission line trips without retraining.
WLS Recovery Error	99.97% Recovery	State tracking MSE drops from 0.007228 p.u.^{2} down to 0.000002 p.u.^{2}.
Dataset Scale	5,000 samples	Solved under non-linear AC power flow laws using `pandapower`.

2. Ground Zero: The Smart Power Grid Explained Simply

Everyday Analogy: The City Water Plumbing System

Imagine a massive metropolitan drinking water network:

- **Generators = Water Reservoirs / Pumping Stations:** Where clean water originates.

- **Buses (Nodes) = Pipe Junctions / Intersections:** Junctions where multiple pipes meet.
- **Branches / Lines (Edges) = Heavy Water Pipes:** High-capacity pipes transporting water between junctions.
- **Loads / Consumers = Neighborhoods / Faucets:** Homes drawing water out of the system.
- **SCADA Sensors (RTUs) = Digital Flow Meters & Pressure Gauges:** Sensors reporting pressure and flow rate every 2 seconds.
- **Energy Management System (EMS) = Central City Water Control Room:** Computer consoles monitoring the entire city to prevent pipe bursts or dry taps.

The Four Core Electrical Telemetry Measurements

At every electrical bus (junction) in an Alternating Current (AC) power grid, Remote Terminal Units (RTUs) report 4 numbers:

1. **Voltage Magnitude (V):** Like water pressure. Standard target is 1.0 per-unit (p.u.).
2. **Voltage Phase Angle (θ):** Like the tilt of a pipe that determines water flow direction. Power flows from higher angle to lower angle.
3. **Active Power (P):** Measured in Megawatts (MW). Real electricity doing physical work (spinning motors, lighting bulbs).
4. **Reactive Power (Q):** Measured in Megavars (MVAr). The invisible magnetic field needed to sustain AC transmission lines and transformers.

The IEEE 14-Bus Test System

In our research, we use the standard **IEEE 14-bus test system**. It represents a portion of the American Midwestern power grid:

- **14 Buses (Nodes):** 5 generator buses (Buses 1, 2, 3, 6, 8) and 9 load/transmission substations.
- **20 Branches (Edges):** High-voltage transmission lines and transformers connecting the buses.
- **56 Total Telemetry Points:** 14 buses $\times$ 4 state measurements $(V, \theta, P, Q) = 56$ features per snapshot.

3. The Cyber Attack: False Data Injection Attacks (FDIA)

Everyday Analogy: The Rogue Accountant & Balanced Ledgers

Why Traditional Security Fails

Suppose a thief steals \$100,000 from a bank safe and leaves. The auditor counts the safe, sees \$100,000 missing, and immediately rings the alarm. That is a **crude attack** (random noise).
Now imagine a **clever rogue accountant**. When stealing \$100,000, the accountant alters invoices, vendor receipts, and tax calculations *simultaneously* across 10 accounting books so that every ledger entry adds up to zero. When the auditor runs standard accounting checks:

$$\text{Total Assets} - \text{Total Liabilities} = 0$$

The system reports zero discrepancy! The auditor stamps "ALL CLEAN" while the bank is bleeding cash. This is exactly what a **False Data Injection Attack (FDIA)** does to a smart power grid.

The Mathematics of State Estimation (WLS) & The Attack Vector

In a standard SCADA control center, the relationship between grid states x (voltages and phase angles) and sensor measurements z is:

$$z = h(x) + e$$

where $h(x)$ represents physical non-linear AC power flow laws, and $e \sim \mathcal{N}(0, \sigma^2)$ is Gaussian sensor noise.

1. Weighted Least Squares (WLS) Estimation: The grid control center estimates state $\hat{x}$ by minimizing measurement residuals:

$$\hat{x} = \arg\min_x [z - h(x)]^T W [z - h(x)]$$

where W is the inverse covariance measurement error matrix.

2. Traditional Bad Data Detection (BDD): The control center calculates the measurement residual:

$$r = \|z - h(\hat{x})\|_2$$

If $r \leq \tau$ (where τ is determined by a Chi-Square χ^2 distribution test), the data is declared normal. If $r > \tau$, an alarm is raised.

3. The Hacker's Secret Formula ($a = Hc$): Under DC power flow approximation, $z = Hx + e$, where H is the grid Jacobian measurement matrix. An attacker with grid topology knowledge constructs an attack vector $a = Hc$, where c is the false state bias injected into the buses. The corrupted telemetry received by the control center is:

$$z_{\text{attack}} = z + a = Hx + Hc + e = H(x + c) + e$$

When the control center calculates the post-attack residual:

$$r_{\text{attack}} = \|z_{\text{attack}} - H\hat{x}_{\text{attack}}\|_2 = \|(Hx + a) - H(\hat{x} + c)\|_2 = \|Hx + Hc - H\hat{x} - Hc\|_2 = \|Hx - H\hat{x}\|_2 = r_{\text{nominal}}$$

The residual is mathematically identical to nominal clean data! The χ^2 test detects **NOTHING**.

The Disaster: The operator believes Bus 5 is running cool, so they route more power through it. In reality, Bus 5 is severely overloaded, causing transmission lines to sag, transformers to catch fire, and protective breakers to trip, triggering a cascading regional blackout.

4. The Solution: Our Graph Digital Twin (GDT) Architecture

Everyday Analogy: The Formula 1 Pit-Lane Simulator

In Formula 1 racing, a car has hundreds of telemetry sensors. In the pit lane, engineers monitor a **live virtual 3D software replica (Digital Twin)** running aerodynamic physics simulations. If a competitor hacks tire sensor #3 to report a false blowout, the Digital Twin cross-verifies suspension strain and velocity physics, flags sensor #3 as spoofed, mutes it, and keeps the car racing safely.

The 4-Pillar GDT Architecture

1. **Spatial Admittance Graph Mirror (Y_{bus}):** Instead of treating telemetry as a flat table of numbers, we map it onto a mathematical graph $G = (V, E, X, E_{\text{attr}})$:
   - Nodes V: 14 substations (buses).
   - Edges E: 20 transmission lines.
   - Node Features $X \in \mathbb{R}^{14\times 4}$: $[V_i, \theta_i, P_i, Q_i]$.
   - Edge Attributes E_{attr}: Physical branch admittance $Y_{ij} = G_{ij} + jB_{ij}$ (conductance and susceptance).
2. **AI Inference Brain (Multi-Task GraphSAGE):** Processes graph neighborhood messages to detect if an attack is happening (Global Head) and identify which buses are hacked (Local Head) in 0.908 ms.
3. **Forensic Explainability Engine (Integrated Gradients):** Computes exact gradient attribution paths to tell the operator which sensor was tampered with.
4. **Closed-Loop Self-Healing Actuator:** Mutes corrupted RTUs and injects rolling historical load forecast baselines (±2% noise), restoring state tracking.

5. Why GraphSAGE? The Architectural Showdown

Why did we choose **GraphSAGE** over Flat MLPs, GCNs, and GATs? This is the most frequently asked question in technical defenses!

Architecture	How It Aggregates	Fatal Flaw / Limitation	Localization F1
Flat MLP (Multi-Layer Perceptron)	Concatenates features into flat 1D vector: $[z_1, z_2, \ldots, z_{56}]$.	Completely ignores graph topology. **Collapses under line outages** ($N-1$ trips) because input dimension changes.	82.20% (outages)
GCN (Graph Convolutional Net)	Normalized Laplacian convolutions: $\sum_u \frac{\mathbf{W}\mathbf{h}_u}{\sqrt{d_v d_u}}$.	**Over-Smoothing Problem:** Uniform neighbor averaging blurs node features into identical mean vectors in dense electrical loops.	70.21% (dense grids)
GAT (Graph Attention Net)	Dynamic attention weights α_{ij} via pairwise node projections.	Excessive quadratic parameter overhead; attention weights fluctuate on static electrical branch admittances.	94.12% (slower)
GraphSAGE (Our Proposed Backbone)	**Explicit Concatenation:** $\mathbf{h}_v = \sigma(\mathbf{W}[\mathbf{h}_v \,\|\, \mathrm{AGG}(\mathcal{N}(v))])$.	**No over-smoothing!** Keeps the node's own state separate from neighbors. Generalizes zero-shot to $N-1$ line trips!	**97.62% ± 0.84%**

The 3-Way Architectural Comparison

The GraphSAGE Mathematical Core Equation

At each layer k, GraphSAGE updates node v's representation through two steps:

$$\mathbf{h}_{\mathcal{N}(v)}^{(k)} = \mathrm{MEAN}\left(\left\{\mathbf{h}_u^{(k-1)}, \forall u \in \mathcal{N}(v)\right\}\right)$$
$$\mathbf{h}_v^{(k)} = \mathrm{ReLU}\left(\mathbf{W}_k \cdot \left[\mathbf{h}_v^{(k-1)} \,\|\, \mathbf{h}_{\mathcal{N}(v)}^{(k)}\right]\right)$$

where $[\mathbf{h}_v \,\|\, \mathbf{h}_{\mathcal{N}(v)}]$ denotes **vector concatenation**. By explicitly isolating the substation's own telemetry from its neighbors, the model retains the unique electrical fingerprint of each bus!

6. Explainable AI (XAI) via Integrated Gradients

Everyday Analogy: Forensic Fingerprint Analysis

When a burglar breaks into a house, a smart alarm doesn't just say "Intruder detected." A forensic investigator dusts for fingerprints, inspects the broken window lock, and confirms: *"The intruder picked lock #3 on the back door."*

In power grids, operators will **never** disconnect a billion-dollar substation based on an opaque AI black box. They demand mathematical proof.

Why Integrated Gradients Instead of SHAP or LIME?

- **The Latency Bottleneck:** SHAP (KernelSHAP) and LIME evaluate models by generating thousands of random perturbations. Calculating SHAP values takes **5 to 15 seconds per sample**. SCADA systems require decisions within **sub-seconds**! SHAP is too slow for real-time cyber defense.
- **Axiomatic Completeness:** Integrated Gradients (Sundararajan et al., ICML 2017) integrates gradients along the straight-line path from a neutral baseline x_0 (clean grid state) to the input x:

  $$\mathrm{Attribution}_i(x) = (x_i - x_{0,i}) \times \int_0^1 \frac{\partial F(x_0 + \alpha(x - x_0))}{\partial x_i} d\alpha$$

  The sum of all attributions precisely equals the difference between the attacked prediction and baseline prediction:

  $$\sum_{i=1}^{M} \mathrm{Attribution}_i(x) = F(x) - F(x_0)$$

  This is computed via a single PyTorch backward pass in **under 2 milliseconds**!

What the Operator Sees: The GDT generates an interactive heatmap showing that Voltage Phase Angle θ at Bus 5 had an attribution weight of **0.486**, proving that Bus 5's phase angle sensor was the attacker's primary manipulation vector.

7. Closed-Loop Active Healing: The Self-Recovery Engine

Most research papers stop at detection ("We found the attack!"). Our framework implements an automated **closed-loop active healing pipeline**:

Detection (GraphSAGE) $\longrightarrow$ **Localization** (Bus 5) $\longrightarrow$ **Mute Bad RTU** $\longrightarrow$ **Inject Load Baseline** $\longrightarrow$ **WLS Recovers**

The 3-Step Self-Healing Process

1. **Step 1: Automated Sensor Quarantine:** The GDT control loop sends an out-of-band signal to SCADA, instructing the Energy Management System to flag and ignore telemetry arriving from Bus 5.
2. **Step 2: Physics-Consistent Baseline Substitution:** Instead of leaving Bus 5 with missing values (which causes WLS matrix singularity), the GDT fetches historical rolling load forecasts ($z_{\text{clean}} \pm 2\%$ noise) based on typical time-of-day power profiles.
3. **Step 3: State Reconstruction & Verification:** The EMS re-runs WLS state estimation with clean surrogate values.

Self-Healing Benchmark Results

- **Under Unmitigated FDIA:** WLS converges to corrupted states without raising alarms. State estimation error is 0.007228 p.u.^{2}.
- **Under GDT Closed-Loop Healing:** State estimation error drops to 0.000002 p.u.^{2} (**99.97% error recovery**).
- **Active Power Tracking:** Grid-wide active power Mean Squared Error (MSE) drops from 51.19 MW^2 down to 21.91 MW^2 (**57.21% recovery**).

8. Codebase Tour: Where Everything Lives

Script / Module	Architecture Role	Key Responsibilities & Functions
`data_generation.py` (in `src/`)	Physical Grid Simulator	Uses `pandapower` to solve non-linear AC power flows across 5,000 states; injects stealthy FDIA vectors $a = Hc$; saves `data/dataset.pt`.
`model.py` (in `src/`)	GNN Backend Brain	Implements `MultiTaskGraphSAGE`, `BaselineMLP`, `BaselineGCN`, and `BaselineGAT`; builds PyG graph batches.
`explainability.py` (in `src/`)	XAI Forensic Engine	Computes Integrated Gradients path integrals across sensor features (V, θ, P, Q) for each substation.
`cyber_mitigation_demo.py` (in `src/`)	Active Healing Engine	Mutes compromised RTUs, substitutes historical load baselines, and benchmarks WLS state recovery.
`visualize.py` (in `src/`)	Operator Dashboard	Renders 3D grid layout charts and feature attribution heatmaps (`figures/grid_anomaly_explanation.png`).
`compare_models.py` (in `experiments/`)	Statistical Benchmark	Executes 5-seed comparative benchmarks across GraphSAGE, MLP, GCN, and GAT models.
`evaluate_topology.py` (in `experiments/`)	Outage Generalization	Simulates $N-1$ transmission line outages (tripped lines) to test zero-shot graph adaptability.
`evaluate_ieee30.py` (in `experiments/`)	Cross-Grid Transfer	Evaluates inductive transfer learning from IEEE 14-bus to IEEE 30-bus feeder network.
`evaluate_wls.py` (in `experiments/`)	Classical WLS Benchmark	Compares GDT against classical industrial WLS state estimation and χ^2 bad data detection.

9. Master Technical Viva & Interview Q&A (Top 12 Questions)

Q1: Why did you build a software Digital Twin rather than using physical hardware?

Answer: "Hardware-in-the-Loop (HIL) testbeds (such as OPAL-RT or Typhoon HIL) cost hundreds of thousands of dollars and are restricted to rigid physical layouts. Our data-driven software Digital Twin solves non-linear AC power flows using `pandapower` under exact Kirchhoff's laws. This allowed us to generate 5,000 physics-conforming states, simulate complex $N-1$ transmission line outages, and rapidly evaluate advanced GNN architectures with sub-millisecond execution."

Q2: Why is reporting 5-seed statistical averages important?

Answer: "In machine learning research, single-run numbers (like a single 100% accuracy result) are often misleading because they can stem from favorable random weight initializations or test set splits. Reporting 5-seed statistical averages ($98.25\% \pm 0.34\%$ Detection F1, $97.62\% \pm 0.84\%$ Localization F1) proves the mathematical stability and reproducibility of our GraphSAGE model across diverse initializations, satisfying rigorous IEEE publication standards."

Q3: How does your model handle transmission line outages ($N-1$ contingencies)?

Answer: "In traditional machine learning, models like MLPs treat inputs as a fixed flat vector ($14 \times 4 = 56$). If a transmission line trips for maintenance, the physical relationship changes, causing MLP localization performance to plunge to 82.20%.
In contrast, GraphSAGE operates directly on the dynamic admittance graph Y_{bus}. When a line trips, we simply update the edge connectivity matrix. GraphSAGE processes the altered graph zero-shot, retaining a **92.80% detection F1 score** without requiring any retraining."

Q4: Why did GraphSAGE outperform GCN and GAT?

Answer: "Standard GCNs average features across all neighbors using a normalized Laplacian matrix. In electrical power grids with dense cyclic loops, repeated averaging causes **over-smoothing**, where node embeddings collapse into identical mean vectors (dropping localization F1 to 70.21%).
GraphSAGE solves this by explicitly **concatenating** the node's previous representation with its aggregated neighbor features: $\mathbf{h}_v = \text{ReLU}(\mathbf{W} \cdot [\mathbf{h}_v \| \mathbf{h}_{\mathcal{N}(v)}])$. This keeps each substation's unique telemetry signature distinct. GAT introduces attention overhead without performance gain because power grid physical branch admittances are fixed physical properties rather than dynamic attention weights."

Q5: Explain how a stealthy FDIA bypasses Chi-Square (χ^2) bad data detection.

Answer: "Classical state estimation computes the measurement residual $r = \|z - H\hat{x}\|_2$. If $r \leq \tau$, data is accepted. An attacker with grid topology access constructs an injection vector $a = Hc$. The corrupted telemetry is $z_{\text{attack}} = Hx + a + e = H(x + c) + e$. When the control center calculates the post-attack residual:

$$r_{\text{attack}} = \|(Hx + a) - H(\hat{x} + c)\|_2 = \|Hx + Hc - H\hat{x} - Hc\|_2 = \|Hx - H\hat{x}\|_2 = r_{\text{nominal}}$$

Because the injected bias algebraically cancels out in the residual calculation, r_{attack} is identical to clean data, completely blinding the Chi-Square detector!"

Q6: Why did you use Integrated Gradients instead of SHAP or LIME?

Answer: "Real-time SCADA operations have strict temporal constraints (scan cycles every 2 to 4 seconds). SHAP and LIME rely on stochastic perturbation sampling, taking 5 to 15 seconds per sample, which is far too slow for real-time cyber defense.
Integrated Gradients computes path integrals via a single backpropagation pass in under 2 milliseconds. Additionally, it satisfies **axiomatic completeness**, meaning the sum of all sensor attributions mathematically equals the exact difference between the model's attacked prediction and clean baseline prediction."

Q7: How does your closed-loop self-healing module recover the grid state?

Answer: "Once GraphSAGE pinpoints the compromised substations (e.g., Bus 5), the GDT isolates telemetry coming from those RTUs. Leaving those buses with missing values would cause matrix singularity in WLS state estimation. Therefore, our module fetches historical rolling load forecasts (±2% noise) matching typical load curves, injects them as surrogate inputs, and re-runs WLS. This reduces state tracking error from 0.007228 p.u.^{2} down to 0.000002 p.u.^{2} (**99.97% error recovery**)."

Q8: How does your research compare against published literature?

Answer: "We benchmarked our GDT against prominent published works:

- **Boyaci et al. (IEEE Trans. Smart Grid 2022):** Evaluated an ARMA-GNN achieving 96.25% detection and 93.91% localization. Our GraphSAGE model achieves **98.25% detection** and **97.62% localization**.
- **Yue & Qu (CARS 2024):** Evaluated GCN models on cyber-physical anomaly detection. Our framework incorporates closed-loop active healing and explainable AI attribution, which their work lacked.

Furthermore, our model runs at sub-millisecond inference latency (0.908 ms), demonstrating full operational feasibility for real-time SCADA deployment."

Q9: How did you test cross-grid scalability (IEEE 30-bus system)?

Answer: "We evaluated inductive transfer learning by training GraphSAGE on the IEEE 14-bus grid and evaluating it on the larger IEEE 30-bus feeder network. Because GraphSAGE learns localized node-to-neighbor aggregation functions rather than a global adjacency matrix, fine-tuning the model with only 200 samples of IEEE 30-bus data restored detection F1 to **90.11

Q10: What is the difference between AC power flow and DC power flow in your simulation?

Answer: "DC power flow is a linearized, simplified approximation: it assumes voltage magnitudes are constant (1.0 p.u.), ignores reactive power ($Q = 0$), and assumes voltage angle differences are very small. While attackers often design FDIAs using the linear DC Jacobian ($a = Hc$), real power grids operate under non-linear AC Kirchhoff laws. We simulated the physical asset using full non-linear AC power flows via `pandapower`, ensuring our models were trained on realistic, physics-conforming telemetry."

Q11: What were the biggest debugging challenges you faced during implementation?

Answer: "Two major technical hurdles:

1. **Exploding Gradients during Multi-Task Training:** The global detection loss and multi-label localization loss had different gradient magnitudes. I balanced them by tuning the loss weighting parameter $\lambda = 0.5$ and incorporating Layer Normalization and residual connections in `src/model.py`.
2. **Data Leakage in Baseline Substitution:** In early healing experiments, substituting clean test states directly resulted in unrealistically perfect reconstruction. I corrected this by introducing ±2% stochastic load forecasting noise to mirror realistic industrial forecasting inaccuracy."

Q12: How does this project directly apply to cybersecurity at Evernorth / Healthcare?

Answer: "While power grids and healthcare systems manage different physical assets, their cybersecurity architectures are mathematically isomorphic:

- In smart grids, SCADA sensors stream voltage and power telemetry; in Evernorth, IoMT devices, pharmacy clearinghouse APIs, and claims servers stream transaction and medical data.
- In both environments, stealthy attackers avoid noisy attacks and blend into legitimate traffic (FDIA in grids $\Longleftrightarrow$ claims manipulation or API credential abuse in healthcare).
- Modeling entities as a graph, enforcing sub-millisecond anomaly detection, demanding explainable root-cause attribution, and automating closed-loop containment are identical core competencies. That is the exact analytical rigor I bring to Evernorth's threat management team!"